TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

HƯỚNG DẪN THỰC HIỆN ĐỀ TÀI MÔN KIỂM THỬ PHẦN MỀM

Giảng viên: Nguyễn Thế Lâm

1. Mục tiêu đề tài

Đề tài cuối kỳ được thiết kế nhằm giúp sinh viên:

- Trực tiếp tiếp cận, đọc hiểu kiến trúc và thiết lập môi trường chạy cho các hệ thống phần mềm thực tế (có độ phức tạp cao, đang được sử dụng rộng rãi).
- Vượt ra ngoài các bài kiểm thử cơ bản để nghiên cứu và triển khai một trong các kỹ thuật kiểm thử nâng cao (Property-Based Testing, Mutation Testing, Fuzzing,...).
- Tổ chức làm việc nhóm theo quy trình phát triển phần mềm chuẩn mực (sử dụng Git, công cụ quản lý Task như Jira/Trello, viết báo cáo kỹ thuật và phân vai rõ ràng).

2. Nội dung thực hiện yêu cầu

2.1. Nội dung đề tài

Mỗi nhóm cần hoàn thành **hai phần bắt buộc** tương ứng với 2 giai đoạn của dự án.

Phần 1: Phân tích hệ thống và thiết lập môi trường (System Analysis & Setup)

Nhóm sẽ chọn 1 dự án (Repo) từ danh sách được cung cấp. Tại bước này, nhóm đóng vai trò như các kỹ sư QA mới tham gia dự án (on-boarding).

- Nhiệm vụ:
  - Đọc hiểu cấu trúc mã nguồn, nhận diện các module/dependency chính.
  - Phân tích và vẽ lại kiến trúc hệ thống, luồng dữ liệu (Data flow) của 3-5 nghiệp vụ trọng tâm.
  - Cài đặt hệ thống chạy thành công trên máy cá nhân (Local) hoặc qua Docker.
  - Lập tài liệu hướng dẫn cài đặt (Môi trường, runtime, biến môi trường, lệnh build/run, seed data).

Phần 2: Ứng dụng kỹ thuật kiểm thử nâng cao (Advanced Testing Implementation)

Nhóm chọn 1 kỹ thuật kiểm thử nâng cao từ danh sách để áp dụng vào hệ thống đã setup ở Phần 1.

- Nhiệm vụ:
  - Nghiên cứu cơ sở lý thuyết, giới hạn và tiêu chí áp dụng của kỹ thuật đã chọn.
  - Thiết kế Kịch bản kiểm thử (Test Model / Test Scenarios / Test Cases) tập trung vào các luồng nghiệp vụ cốt lõi hoặc module tiềm ẩn nhiều rủi ro nhất.
  - Triển khai viết mã kiểm thử tự động (Automation Script / Harness).
  - Chạy kiểm thử, thu thập Log/Metrics, phát hiện lỗi (Defect) hoặc chứng minh độ bao phủ (Coverage/Score), sau đó tiến hành phân tích nguyên nhân cốt lõi (Root Cause Analysis).

2.2. Quy định về nhóm

Quy mô nhóm: 3 - 5 sinh viên/nhóm.
Hình thức: Làm việc nhóm theo mô hình Agile/Scrum. Mỗi nhóm đóng vai trò là một "Đội Đảm bảo Chất lượng"(QA Team) độc lập, tiếp nhận và kiểm thử một dự án mã nguồn mở thực tế.
Nguyên tắc chọn đề tài: Hai nhóm không thể chọn chung cùng một tổ hợp [Dự án (Repo) + Kỹ thuật kiểm thử]. Mỗi tổ hợp Repo + Kỹ thuật chỉ cho phép tối đa 01 nhóm đăng ký thực hiện.

Cách thức đăng ký:

- Các nhóm gửi đề tài đăng ký theo mẫu (mỗi nhóm gửi tối đa 3 đề tài là 3 nguyện vọng ưu tiên) vào email: lam.th.ngn@gmail.com.
- Trường hợp 2 nhóm chung 1 đề tài, đề tài gửi sớm hơn sẽ được chấp nhận trước, nhóm còn lại sẽ chuyển xuống nguyện vọng 2 hoặc 3 nếu vẫn bị trùng.

3. Cấu trúc cơ bản cho mọi tổ hợp/đề tài

Để đảm bảo các nhóm được đánh giá trên cùng một khung tiêu chuẩn, dù chọn tổ hợp Repo và Kỹ thuật nào, các nhóm cũng phải thực hiện và trình bày báo cáo cơ bản theo cấu trúc này:

- **Phần A:** Mô tả mục tiêu hệ thống, actor/use case chính, cây thư mục/module, dependency và luồng dữ liệu.
- **Phần B:** Vẽ ít nhất 01 sơ đồ kiến trúc ở mức component/container và giải thích 3-5 module liên quan trực tiếp tới phần kiểm thử.
- **Phần C:** Tài liệu hóa môi trường, phiên bản, lệnh build/run, DB/service, seed data; cung cấp bằng chứng hệ thống chạy thành công và ít nhất 03 luồng nghiệp vụ chạy được.
- **Phần D:** Trình bày nguyên lý, lý do chọn phạm vi, giả định, rủi ro và tiêu chí pass/fail của kỹ thuật được chọn.
- **Phần E:** Thiết kế Test Model/Scenario/Case hoặc Artefact tương đương theo kỹ thuật; chỉ rõ input, precondition, expected result/invariant và dữ liệu test.
- **Phần F:** Chứa mã nguồn test/harness/script có thể chạy lại tự động; cung cấp file README chứa lệnh chạy và dependency.

- **Phần G:** Log/report/metrics, danh sách lỗi (defect) hoặc hành vi bất thường, và phân tích nguyên nhân; Nếu không tìm thấy lỗi, phải chứng minh coverage/score/threshold đã đạt.
- **Phần H:** Lưu cứng (pin) commit/tag của repo, lưu cấu hình test và mô tả cách để người khác có thể chạy lại test trên máy mới.

4. Lịch trình và các cột mốc (Milestones)

Dự án được quản lý theo dạng các Sprint. Các mốc thời gian cụ thể:

Mốc thời gian	Sự kiện	Yêu cầu cần đạt (Deliverables)
18/9	**Hạn chót Đăng ký Đề tài**	Nhóm chốt danh sách thành viên, Repo đã chọn và Kỹ thuật kiểm thử sẽ áp dụng. Chốt Commit SHA/Tag của Repo để cố định phiên bản. Thiết lập xong công cụ quản lý dự án (Trello/Jira).
20/10	**Báo cáo Giữa kỳ (Milestone 2)**	Demo hệ thống chạy thành công trên máy tính. Trình bày sơ đồ kiến trúc, tài liệu on-boarding. Báo cáo kế hoạch áp dụng kỹ thuật kiểm thử (Phạm vi test, giả định, tiêu chí đánh giá).
Ngày thi cuối kỳ theo lịch nhà trường (thường là tháng 12)	**Bảo vệ Cuối kỳ (Final Defense)**	Thuyết trình toàn bộ dự án. Demo chạy script automation. Nộp toàn bộ mã nguồn kiểm thử và Báo cáo tổng kết. *(Lưu ý: Báo cáo nộp trước khi bảo vệ 1 tuần)*. Đánh giá chéo (Peer-review) giữa các thành viên.

LƯU Ý: Sinh viên lưu ý quản lý thời gian chặt chẽ. Hệ thống mã nguồn mở rất đồ sộ, hãy **giới hạn phạm vi (Scope) phù hợp** (ví dụ: chỉ kiểm thử module thanh toán, luồng duyệt tài liệu...) thay vì ôm đồm toàn bộ hệ thống.

5. Quy trình làm việc

Bài tập lớn khuyến khích các nhóm làm việc có tổ chức và phối hợp nhịp nhàng:

- Nhóm nên tự phân chia công việc hợp lý (ví dụ: người lo cài đặt môi trường, người viết kịch bản, người code test...).
- Nên sử dụng các công cụ như Trello, Jira hoặc Github Projects để dễ dàng theo dõi tiến độ công việc chung.
- Khuyến khích dùng Git để quản lý mã nguồn. Các ghi chú quan trọng hoặc hướng dẫn chạy code nên được viết tóm tắt trong file README để các thành viên khác dễ nắm bắt.

6. Yêu cầu báo cáo và tiêu chí đánh giá đề tài nhóm

Điểm của đề tài sẽ cân nhắc cả **sản phẩm đạt được** và **sự nỗ lực, quá trình phối hợp** của nhóm. Dưới đây là một số tiêu chí tham khảo cho báo cáo và buổi bảo vệ:

A. Đánh giá báo cáo chuyên môn và mã nguồn

- Báo cáo trình bày được mục tiêu hệ thống, vẽ lại được luồng dữ liệu cơ bản ở mức dễ hiểu.
- Có hướng dẫn cài đặt môi trường rõ ràng để giảng viên có thể dựng lại và chạy thử (khuyến khích dùng Docker nếu có thể).
- Cho thấy nhóm hiểu được kỹ thuật kiểm thử đang áp dụng và biết cách vận dụng vào dự án.
- Các kịch bản kiểm thử (scenarios) có mô tả cụ thể. Mã nguồn test nên gọn gàng, ưu tiên có thể chạy tự động và sinh ra kết quả.
- Từ kết quả chạy test, nhóm có thể chỉ ra được lỗi phần mềm (nếu có) hoặc giải thích được ý nghĩa của các log/kết quả thu được.

B. Đánh giá quản trị dự án và thuyết trình

- Có lịch sử công việc và commit (Git) để cho thấy sự tham gia đều đặn của các thành viên.
- Slide báo cáo có bố cục gọn gàng. Sinh viên trình bày tự tin, dễ hiểu và có sự phối hợp tốt khi báo cáo.
- Cố gắng trả lời đúng trọng tâm các câu hỏi góp ý từ giảng viên.

C. Đánh giá đóng góp cá nhân (Peer Evaluation)

- Cuối kỳ, nhóm sẽ có phần đánh giá chéo mức độ đóng góp của từng thành viên. Điểm cá nhân có thể được điều chỉnh đôi chút so với điểm chung dựa trên mức độ tham gia thực tế vào bài tập lớn.

Phụ lục 1: danh sách repo mã nguồn mở

1.1. Gợi ý chọn repo theo năng lực nhóm

- **Nhóm mới tiếp cận dự án mã nguồn mở:** R01 Spring PetClinic, R02 Mealie, R03 Snipe-IT, R04 nopCommerce.
- **Nhóm khá, có kinh nghiệm Docker/web/API:** R05 Cal.com, R06 Directus, R07 Outline, R08 Paperless-ngx, R09 Chatwoot.
- **Nhóm mạnh, có thể đọc monorepo/multi-service:** R10 Twenty, R11 Plane, R12 Saleor, R13 Immich, R14 Jellyfin, R15 Mattermost, R16 Appsmith.

1.2. Các lưu ý khi chốt repo

- Khuyến nghị đối với repo quy mô lớn (R09 - R16): Nên khóa phiên bản theo release/tag và cho phép nhóm dùng Docker/Docker Compose. Không nên yêu cầu sinh viên hiểu toàn bộ codebase; chỉ cần phân rã 3 - 5 module hoặc luồng chính có liên quan trực tiếp tới phần kiểm thử.
- Nên chốt tag/release cụ thể và lưu lại commit SHA trong đề tài để tránh main branch thay đổi giữa học kỳ gây vỡ test.
- Yêu cầu nhóm tự lập tài liệu cài đặt từ máy sạch (phiên bản runtime, DB, Docker, biến môi trường, port, seed data và lệnh chạy).
- Cho phép giới hạn phạm vi, ví dụ chỉ kiểm thử module booking, checkout, asset checkout, document workflow hoặc một nhóm API.
- Một số dự án dùng AGPL/BSL/custom license. Sinh viên cần đọc kỹ file LICENSE nếu có ý định phân phối lại bản build hoặc mã sửa đổi. Với môn học, nên ưu tiên clone/run/test trong môi trường học tập nội bộ.

1.3. Danh sách chi tiết các repo

Mã	Repo / Lĩnh vực	Stack & Kiến trúc	Độ khó	Nhãn	Kỹ thuật	Ghi chú
R01	**spring-projects/spring-petclinic** Quản lý phòng khám thú y	Java, Spring Boot, Thymeleaf; MySQL/PostgreSQL; Testcontainers	Dễ - vừa ⋆ ○ ○	API, WF, DATA	K01, K03, K06, K08, K16	Rất phù hợp cho nhóm lần đầu đọc mã nguồn mở.
R02	**mealie-recipes/mealie** Quản lý công thức, thực đơn	Python backend + Vue/TypeScript frontend; REST API	Dễ - vừa ⋆ ○ ○	UI, API, WF, DATA	K01, K05, K06, K10, K11	Phạm vi chức năng vừa phải, dễ xây dựng test scenario.
R03	**grokability/snipe-it** Quản lý tài sản CNTT	PHP, Laravel 12; DB quan hệ; Docker	Vừa ⋆ ⋆ ○	UI, API, WF, DATA, SEC	K06, K08, K10, K15, K16	Tốt cho kiểm thử phân quyền, dữ liệu biên.

Mã	Repo / Lĩnh vực	Stack & Kiến trúc	Độ khó	Nhãn	Kỹ thuật	Ghi chú
R04	**nopSolutions/nopCommerce** Thương mại điện tử	C#, ASP.NET Core, SQL Server; kiến trúc plugin	Vừa ⋆⋆∘	UI, API, WF, DATA, PERF, SEC	K06, K07, K08, K10, K15	Nghiệp vụ phong phú; phù hợp kiểm thử tổ hợp, hiệu năng.
R05	**calcom/cal.com** Hạ tầng đặt lịch/scheduling	TypeScript, Next.js, PostgreSQL; monorepo	Vừa - khó ⋆⋆∘	UI, API, WF, DATA, CONC	K02, K05, K06, K10, K15	Đặc biệt phù hợp kiểm thử stateful và concurrency.
R06	**directus/directus** Headless CMS / data platform	TypeScript/Node.js, Vue; REST & GraphQL; Docker	Dễ - vừa ⋆⋆∘	UI, API, DATA, SEC, PERF	K04, K05, K07, K08, K16	Dễ dựng; lý tưởng cho API schema testing, phân quyền.
R07	**outline/outline** Knowledge base cộng tác	TypeScript, React, Node.js; PostgreSQL/Redis	Vừa - khó ⋆⋆∘	UI, WF, DATA, REAL-TIME	K02, K10, K11, K15, K16	Tốt cho kiểm thử workflow tài liệu, phân quyền, visual.
R08	**paperless-ngx/paperless-ngx** Quản lý tài liệu, OCR	Python/Django + Angular; OCR/ML pipeline	Vừa ⋆⋆∘	UI, API, DATA, FILE, ASYNC	K01, K05, K09, K12, K16	Phù hợp fuzzing file/input, resilience pipeline.
R09	**chatwoot/chatwoot** Customer support đa kênh	Ruby on Rails + Vue; PostgreSQL/Redis; realtime	Vừa - khó ⋆⋆⋆	UI, API, WF, REALTIME, DIST	K02, K04, K07, K12, K15	Nhiều state, event và concurrency để tạo test sâu.
R10	**twentyhq/twenty** CRM hiện đại	TypeScript, NestJS, React, PostgreSQL; monorepo	Khó ⋆⋆⋆	UI, API, WF, PERF, CONC	K04, K05, K07, K10, K15	Tốt cho nhóm mạnh TypeScript; kiến trúc đủ lớn.
R11	**makeplane/plane** Quản lý dự án/issue/sprint	TypeScript + Python/Django; PostgreSQL, Redis	Khó ⋆⋆⋆	UI, API, WF, DIST, CONC	K02, K04, K07, K12, K15	Tốt cho model-based testing (workflow quản lý issue).
R12	**saleor/saleor** Headless commerce API	Python/Django, GraphQL; API-first	Khó ⋆⋆⋆	API, WF, DATA, PERF, SEC	K04, K05, K07, K08, K15	Mạnh back-end/API; kiểm thử GraphQL, webhook.

Mã	Repo / Lĩnh vực	Stack & Kiến trúc	Độ khó	Nhãn	Kỹ thuật	Ghi chú
R13	**immich-app/immich** Quản lý ảnh/video tự host	TypeScript/NestJS + Svelte; PostgreSQL, ML service	Khó ⋆⋆⋆	UI, API, FILE, DIST, PERF, CONC	K01, K09, K12, K13, K15	Nổi bật nhưng nặng tài nguyên, cần cấu hình máy tốt.
R14	**jellyfin/jellyfin** Media server backend	C#/.NET, ffmpeg, REST/API	Vừa - khó ⋆⋆⋆	API, FILE, PERF, DIST	K05, K07, K09, K12, K14	Tốt cho hiệu năng, fuzzing media; cần lưu ý ffmpeg.
R15	**mattermost/mattermost** Nền tảng chat cho team dev	TypeScript + Go; monorepo; realtime	Khó ⋆⋆⋆	UI, API, REAL-TIME, DIST, PERF, CONC	K04, K07, K12, K15, K16	Quy mô lớn, kiểm thử hệ thống production-grade.
R16	**appsmithorg/appsmith** Low-code platform	TypeScript/Java; frontend + backend; sandbox	Khó ⋆⋆⋆	UI, API, DIST, PERF, SEC	K04, K07, K08, K10, K15	Codebase lớn; dành cho nhóm có năng lực đọc monorepo.

Phụ lục 2: danh sách kỹ thuật kiểm thử nâng cao

Danh sách các kỹ thuật có thể sinh ra các kịch bản test (artefact) cụ thể, chạy được tự động:

Mã	Kỹ thuật	Ý tưởng cốt lõi	Đối tượng phù hợp	Công cụ	Đầu ra	Khó
K01	**Property-Based Testing**	Xác định tính chất đúng (invariant); sinh nhiều dữ liệu, shrink ca lỗi.	Hàm xử lý, validation, API có input phong phú.	Hypothesis, jqwik, fast-check	≥ 3 property; sinh test tự động; lưu counterexample.	⋆⋆∘
K02	**Model-Based / Stateful**	Mô hình hóa hệ thống bằng state/transition; tự sinh chuỗi hành động.	Booking, issue workflow, chat, checkout.	GraphWalker, Hypothesis	Sơ đồ State; coverage trạng thái; ≥ 3 chuỗi nhánh.	⋆⋆⋆
K03	**Mutation Testing**	Chèn "mutant"vào mã nguồn; bộ test tốt phải phát hiện và làm mutant thất bại.	Module có unit test ổn định, business logic.	Stryker, PIT, mut-mut	Baseline tests; báo cáo killed/survived; so sánh score.	⋆⋆∘
K04	**Contract Testing**	Contract sinh từ consumer và replay tại provider để tránh breaking change.	Frontend-backend, microservice, API.	Pact	Xác định consumer-provider; ≥ 5 interactions; demo.	⋆⋆⋆
K05	**Schema-Based API / Fuzz**	Dùng OpenAPI/GraphQL để sinh request hợp lệ/lỗi và test invariant/status.	Repo API-first, có OpenAPI/GraphQL.	Schemathesis, Postman	Chạy schema tests; lưu reproducer; thống kê lỗi.	⋆⋆∘
K06	**Combinatorial / Pairwise**	Giảm không gian tổ hợp bằng pairwise thay vì vét cạn tích Descartes.	Form nhiều tham số, role, cấu hình, filter.	Microsoft PICT	Sinh pairwise; so sánh với exhaustive; chạy mẫu.	⋆∘∘
K07	**Performance (Load/Stress)**	Đo latency, throughput, error rate dưới tải thực tế và cực hạn.	API, chat, commerce, media, nhiều user.	k6, JMeter, Gatling	Workload model; ≥ 3 profile tải; biểu đồ RPS.	⋆⋆∘
K08	**DAST (Dynamic Security)**	Chủ động gửi payload tấn công để tìm lỗ hổng web phổ biến.	Web app có auth, API, form, session.	OWASP ZAP	Chỉ quét local; passive/active scan; verify thủ công.	⋆⋆∘
K09	**Fuzz Testing**	Sinh input tốc độ cực lớn để tìm crash, hang, memory bug, lỗi parser.	Parser, file xử lý, import/export.	libFuzzer/AFL++/Jazzer	Fuzz target; corpus ban đầu; minimized input.	⋆⋆⋆

Mã	Kỹ thuật	Ý tưởng cốt lõi	Đối tượng phù hợp	Công cụ	Đầu ra	Khó
K10	**Visual Regression Testing**	So sánh screenshot với ảnh gốc (baseline) để tìm thay đổi UI ngoài ý muốn.	Web UI ổn định, dashboard, editor.	Playwright	Có baseline; $\geq$ 8 màn hình; intentional vs regression.	⋆ ○ ○
K11	**Automated Accessibility**	Tự động kiểm tra WCAG, kết hợp kiểm tra dùng bàn phím thủ công.	Web UI, form, modal, navigation.	axe-core, Playwright	Chạy axe; phân nhóm lỗi; kiểm tra keyboard.	⋆ ○ ○
K12	**Fault Injection / Resilience**	Gây latency, network down để kiểm tra retry, circuit-breaker, fallback.	Ứng dụng nhiều dependency (DB, Redis...).	Toxiproxy, Docker Compose	Lập Fault model; $\geq$ 4 scenarios; automated injection.	⋆ ⋆ ⋆
K13	**Metamorphic Testing**	Kiểm tra quan hệ nhiều lần chạy sau khi biến đổi input (không cần oracle).	Search, data transform, recommendation.	Tự xây, Hypothesis	$\geq$ 3 metamorphic relations; tự động so sánh.	⋆ ⋆ ⋆
K14	**Differential Testing**	Chạy 1 input trên 2 version/config và tìm ra khác biệt hành vi.	API version, DB backend, bản cũ/mới.	Tự xây, Hypothesis	Hai môi trường test; phân loại danh sách lỗi defect.	⋆ ⋆ ⋆
K15	**Concurrency / Race**	Tạo nhiều request đồng thời để tìm lỗi double booking, lost update.	Booking, cart, chat, realtime, inventory.	k6, Playwright parallel	Xác định invariant; $\geq$ 3 race scenarios.	⋆ ⋆ ⋆
K16	**Ephemeral Integration**	Khởi tạo dependency thật trong Docker (tránh mock sai lệch) cho test.	Backend có DB/cache/broker.	Testcontainers, Docker	Fixture container; $\geq$ 5 integration tests.	⋆ ⋆ ○

Phụ lục 3: quy tắc ghép tổ hợp repo × kỹ thuật

Không phải Repo nào cũng phù hợp với mọi Kỹ thuật. Sinh viên cần tham khảo bảng quy tắc dưới đây để chọn tổ hợp khả thi:

Kỹ thuật	Nên ghép khi	Tránh hoặc điều kiện không nên ghép
K01 Property-based	Ưu tiên module có hàm xử lý logic, validation rõ ràng.	Không ép vào UI thuần hoặc quy trình workflow quá phụ thuộc môi trường.
K02 Model-based	Ưu tiên R05, R07, R09, R11, R12 hoặc workflow có lưu trạng thái.	Không dùng nếu nhóm chỉ kiểm thử một endpoint CRUD đơn giản.

Kỹ thuật	Nên ghép khi	Tránh hoặc điều kiện không nên ghép
K03 Mutation	Repo/module phải có unit test ổn định, build nhanh.	Tránh test toàn bộ monorepo rất lớn; giới hạn lại 1 package/module.
K04 Contract	Phải có ranh giới API rõ ràng giữa phía consumer - provider.	Nếu repo monolith không có client/provider tách biệt thì không áp dụng.
K05 Schema-based	Dự án có sẵn file OpenAPI/GraphQL hoặc có thể trích schema.	Không phù hợp nếu không có API (vd: công cụ desktop/media local).
K06 Pairwise	Dự án có form điền nhiều tham số, cấu hình, bộ lọc tìm kiếm.	Dễ bị quá đơn giản nếu chỉ sinh case ra Excel mà không code script chạy.
K07 Performance	Chọn được luồng đọc/ghi đại diện và có sẵn script tạo dữ liệu tải.	Chỉ load test ở local/lab; tuyệt đối không bắn tải vào public server/demo.
K08 DAST	Ưu tiên trang có luồng đăng nhập, form phân quyền, session.	Chỉ quét instance cài ở máy Local. Cấm quét các hệ thống Internet.
K09 Fuzz	Ưu tiên parser, xử lý file tải lên, chức năng Import/Export.	Cần tạo bộ Harness rõ ràng; tránh việc fuzz mù mờ toàn bộ hệ thống.
K10 Visual regression	UI ổn định và có thể cố định được kích thước browser/data.	Không tốt với trang realtime đổi dữ liệu liên tục nếu không làm giả (mock).
K11 Accessibility	Ưu tiên UI có chứa form, modal, menu điều hướng, dashboard.	Bắt buộc bổ sung phần tự tay kiểm tra phím (keyboard), không chỉ chạy tool.
K12 Fault injection	Hệ thống phụ thuộc nhiều qua mạng (gọi DB, Redis, Worker node).	Không cần thiết với ứng dụng chạy 1 tiến trình độc lập không gọi dịch vụ.
K13 Metamorphic	Có thể xác định quan hệ biến đổi dữ liệu (vd: sắp xếp, bộ lọc).	Không dùng nếu nhóm không hiểu và không giải thích được "Oracle Problem".
K14 Differential	Cần 2 phiên bản hoặc 2 cấu hình khác nhau dùng chung hợp đồng API.	Phải lọc được khác biệt hợp lệ trước khi gán nhãn đó là lỗi (defect).
K15 Concurrency	Ưu tiên chức năng đặt chỗ (booking), giỏ hàng, kho hàng, chat.	Phải kiểm tra tính nhất quán dữ liệu ở CSDL, không chỉ gọi API mù.
K16 Testcontainers	Dự án Backend kết nối với DB/cache/message broker.	Chú ý giới hạn số lượng dependency để máy cá nhân sinh viên chạy được.

Phụ lục 4: nguồn tham khảo

4.1. Link kho chứa mã nguồn (GitHub Repositories)

- **R01:** https://github.com/spring-projects/spring-petclinic
- **R02:** https://github.com/mealie-recipes/mealie
- **R03:** https://github.com/grokability/snipe-it
- **R04:** https://github.com/nopSolutions/nopCommerce
- **R05:** https://github.com/calcom/cal.com
- **R06:** https://github.com/directus/directus
- **R07:** https://github.com/outline/outline
- **R08:** https://github.com/paperless-ngx/paperless-ngx
- **R09:** https://github.com/chatwoot/chatwoot
- **R10:** https://github.com/twentyhq/twenty
- **R11:** https://github.com/makeplane/plane
- **R12:** https://github.com/saleor/saleor
- **R13:** https://github.com/immich-app/immich
- **R14:** https://github.com/jellyfin/jellyfin
- **R15:** https://github.com/mattermost/mattermost
- **R16:** https://github.com/appsmithorg/appsmith

4.2. Tài liệu & công cụ kỹ thuật kiểm thử

- **Property-Based Testing - Hypothesis:** https://hypothesis.readthedocs.io/
- **Model-Based Testing - GraphWalker:** https://graphwalker.github.io/
- **Mutation Testing - Stryker Mutator:** https://stryker-mutator.io/docs/
- **Contract Testing - Pact:** https://docs.pact.io/
- **Schema-Based API - Schemathesis:** https://schemathesis.github.io/schemathesis/
- **Pairwise Testing - Microsoft PICT:** https://github.com/microsoft/pict
- **Performance Testing - k6:** https://grafana.com/docs/k6/latest/
- **DAST - OWASP ZAP:** https://www.zaproxy.org/docs/
- **Fuzz Testing - OSS-Fuzz:** https://github.com/google/oss-fuzz
- **Visual Regression - Playwright:** https://playwright.dev/docs/test-snapshots
- **Accessibility Testing - axe-core:** https://github.com/dequelabs/axe-core

- **Fault Injection - Toxiproxy:** https://github.com/Shopify/toxiproxy
- **Ephemeral Integration - Testcontainers:** https://testcontainers.com/